AI report: DD2 Project 1, Option 2 (group gb2)

Abdelrahman Rashed (900231248) · Ziad Khairaldin (900225292) · AI used as a **development aid** (overview 4.1): no AI model runs when the tool runs, and the same input always gives the same output. The evidence for everything below is in the repository: `AGENTS.md`, `ai/use_log.md` (rows L-001 to L-033), `ai/mistakes_log.md` (M-01 to M-20), and the `AI-assisted:` trailer of every assisted commit.

1. What the AI did and what we did, part by part

Part	What we did	What the AI did
Rules (`AGENTS.md`)	Decided what is checked and what the assistant may not touch; wrote it before any code	Drafted the wording from our list (L-001)
Truth tables, classes, Liberty reader (`tt.py`, `boolexpr.py`, `liberty.py`)	Fixed the bit convention (Appendix A), chose which counts must come out (3,984 P, 222 NPN, 428 HD cells)	Wrote the code (L-002, L-003)
AIG and cuts (`make_aig.sh`, `aiger.py`, `cuts.py`)	Specified the check (simulate every cut; leaves must separate the root); compared AIG sizes with Table 2	Wrote the reader, the enumerator and the check (L-005, L-006)
Redundancy checker and Table 4 self-test (`redundancy.py`, `cmos.py`)	Read section 5 and Table 3, decided the tests, investigated the 6-vs-5 row by hand	Wrote the checker and the self-test printout (L-004, L-011)
Table 1 and the flow (`table1.py`, Makefile)	Ran it against the handout, found the Yosys build issue with the TA, chose the pack image	Wrote the runner; fixed paths (L-007 to L-010, L-013)
Cost and area model, ranking (`costmodel.py`, `candidates.py`)	Ziad: designed the exclusive-cone saving, the cap, Table 3 exceptions; hand-costed nand2, and2, a21oi, and4	Wrote the code and its tests (L-022 to L-024)
Requests, Liberty, mapping, equivalence (`write_*.py`, `map_*.py`, `check_mapping_equivalence.py`)	Decided the checks (189 + N, function per cell, cec/dsec against the RTL), wrote the order of operations	Wrote the code, broken cases inside the test files (L-014 to L-021)
Greedy set, curve, final table, 30%, ISA (`select_final_set.py`, `final_*.py`, `sensitivity.py`, `isa_compare.py`)	Ziad: the keep rule (> 0.05 um2, rounding of Yosys's printout), which cell to scale	Wrote the code and tests (L-025 to L-027)
Interview demo, OpenSTA delay, real cells (`demo.py`, `worst_delay.py`, `class_counts.py`)	Decided the demo runs every step on the new design alone and then with our final set; chose the STA constraints	Wrote the code; built the tools image with OpenSTA (L-028 onwards)
Reports	Chose the analyses and checked every number against `results/`	Drafted text from the result files

2. Tools, models and cost

- Claude (Anthropic) `claude-opus-5-5`, in the Claude app and Claude Code (Abdelrahman, checkpoints 1 to 3).
- Claude `claude-sonnet-5-5` in Claude Code, and Claude in chat (Ziad, checkpoints 2 and 3).
- Cost: both members used flat-rate Claude subscriptions (Claude app / Claude Code); no per-request API charges, so no per-task cost breakdown exists. The time cost is in section 7.
- No model runs inside the tool; the Makefile, scripts and tests contain no network or model call (AGENTS.md rule 4).

3. How we described tasks, and how we checked the output

Each request named the handout section, the exact command or table to reproduce, and the check that decides pass or fail, for example: "reproduce Table 1 with the commands of section 2; it passes only if every area equals the handout at 0.1 um2". `AGENTS.md` was loaded in every session: the target numbers (Tables 1, 2 and 4) must be reproduced, not adjusted, and the assistant may not edit `tests/broken/`, the check scripts or the PDK.

A typical exchange, for the interview demo (L-028):

- **Request.** "The interview runs the whole flow from a clean checkout of the tag on a design given on the day (handout 9.4). `make demo` only runs checkpoint 1. Make it run every step on that design, reusing the code `make all` and `make final` use; nothing in `results/` may change."
- **What came back.** `src/demo.py` calling the existing step functions, plus a way to register a design that is not in Table 2.
- **What we checked.** `make demo` on our small test design and on pic1826, which the flow had never analysed: the cut check, the 189 + N Liberty count and every equivalence check had to pass, and `git status` had to show `results/` unchanged.
- **What we changed.** The old `make demo` target passed no `read_verilog` flags, and pic1826 needs `-sv` (handout section 3); the target now takes `FLAGS`, and `IN` may be a file, a folder or several files.

We checked output in three ways, in this order of trust:

1. **Against the handout's numbers.** Table 1 (24 areas), Table 2 (AND counts), Table 4 (class counts). These are independent of anything the AI wrote.
2. **With a check that has a broken case.** Every check in `make test` must fail on a deliberately broken input (section 3 of the technical report); a check whose broken case passes is itself wrong (M-07, M-08).
3. **By reading the output.** Header lines of AIG files, Yosys logs, demo printouts (M-01, M-02, M-06).

4. How we applied the rules of section 4.3

1. *Pass or fail from check scripts.* `make test`, `make check`, the Table 1 comparison, the 189 + N import count and ABC cec/dsec decide; no step asks a model.
2. *Every number from a file.* Each table in the report names its file in `results/`. When the report needed the number of classes over all designs, we added `src/class_counts.py` to the flow (`results/class_counts.csv`) rather than quote a one-off computation.
3. *A broken case per check.* Listed in the technical report, section 3, and in README section 7.
4. *AI does not change checks, broken cases or the PDK.* The broken cases in `tests/broken/` were written by us; broken cases the AI wrote live inside new test files instead (L-012). Two violations happened and are logged: an assistant added `tests/broken/wrong_cost_expression.json` (M-15), and during checkpoint 3 an assistant removed it again and moved the case into `tests/test_candidates.py`; both are disclosed here, and the check still has its broken case. The PDK is linked read-only, never committed.
5. *Every member can explain every line.* Functions are short, each with a docstring naming the handout step it implements (AGENTS.md, "When an assistant helps").

5. The five most instructive AI mistakes

1. **M-04, the wrong Yosys build.** The AI ran Table 1 with `yowasp-yosys` 0.33 because it prints the same version and git hash as native 0.33. Script A differed by up to 6.6%. Script B matched on four designs, so the gap looked like noise. Caught by the TA rerunning our scripts with native Yosys; now `make check` fails unless the Yosys is 0.33, and Table 1 must match at 0.1 um2. *A weaker check (a version string) passes both builds.*
2. **M-05 / M-01, the AIG did not match Table 2.** The AI first removed `-zinit` (M-01, reasoning that it adds inputs) and added an `opt_clean`; our AIGs had exactly 3 fewer AND nodes per flip-flop than Table 2. Caught only by comparing with Table 2. *The cut check passes on any AIG; it cannot see that the graph itself is not the handout's.*
3. **M-08, Yosys accepted a broken Liberty file.** The AI's broken case assumed Yosys rejects a file with a missing brace. Yosys 0.33 read it without error and imported 190 cells instead of 200. Caught because the broken-case test failed. *A check that only looks for a Yosys error passes this file; the 189 + N count catches it.*

4. **M-09, the equivalence check failed the baseline.** The first setup compared the netlist with the raw RTL and paired flip-flops by order, and reported even the baseline cpu8 netlist (no new cells) as not equivalent. Caught because we ran the check on a netlist that must pass first. *Run only on new-cell netlists, the failures would have been blamed on the new cells.*
5. **M-14, the saving was overestimated 4x.** The saving counted every node of an unoptimized AIG cone; the top candidate looked worth 985 um2 instead of 250. All tests passed. Caught because the top cell needed only 3 ANDs but its cones averaged more. *No unit test could see it; only remapping (steps 9 and 10) measures the real saving, and it confirmed the ranking is rough (the first-ranked cell made the total larger).*

All 19 logged mistakes, by what caught them

Caught by	Mistakes
A handout number (Tables 1, 2, 4)	M-04 (wrong Yosys build), M-05 (`-zinit` and `opt_clean` deviated from the handout), M-01 (extra AIG inputs)
A broken case failing to fail	M-08 (Yosys accepts a broken Liberty), M-07 (check crashed instead of failing)
Running the check on an input that must pass	M-09 (equivalence setup failed the baseline netlist)
Reading the output against expectations	M-02 (constant cuts ranked), M-06 (dangling choice nodes), M-10 (and4 costed 14T), M-11 (inv_1 classed multi-stage), M-14 (saving 4x overestimated), M-18 (OpenSTA reset path read as worst delay)
Re-reading the rules or the handout	M-12 (invented fallback area), M-13 (Table 3 exceptions dropped), M-15 (assistant edited `tests/broken/`)
The machine itself	M-03 (WASI Yosys cannot follow symlinks), M-16, M-17 (report drafts misquoted results), M-19 (generator filled the disk and starved its own SPICE jobs), M-20 (curve library truncated before it was read)

Half of these produced code that ran cleanly and passed its own tests. What caught them was outside the code: the handout's numbers, a broken case, or a human reading the output.

6. A decision we would never hand to an AI

Whether a measured number "matches" the handout. When Table 1 or Table 4 does not match, the easy fix is to change the comparison, the tolerance or the target, and an assistant optimizing for a passing test will propose it. We keep the targets in one place (`src/designs.py` TABLE1, `redundancy.py` Table 4 rows) marked "do not edit", and the check compares at the handout's precision. The check that catches an AI getting it wrong: `git diff` on those constants must be empty, and `make table1` must print TABLE 1 REPRODUCED from a clean checkout. The Table 4 row we do not reproduce (6 vs 5) stays a reported mismatch, not an adjusted target.

7. Where AI saved time and where it cost time

Saved: boilerplate that is easy to check: the AIGER reader, Liberty reader and writer, CSV/Markdown tables, Makefile targets; precomputing canonical forms for 65,536 tables; enumerating Table 3's tests; the demo that strings existing steps together. **Cost:** the Yosys build (M-04) took a checkpoint to find; the `-zinit` reversal (M-01, M-05) cost a full regeneration of every result; broken cases that did not break (M-08) and checks that crashed instead of failing (M-07) each needed a second round; the saving model (M-14) looked right and passed tests while being 4x off.

Step 11 (real cells) is the clearest example of both at once. The AI assembled the tools image (OpenSTA from source, the generator's pinned Python packages, and the exact Magic and Netgen commits the generator was qualified with) in one evening, which saved perhaps a day of by-hand tool building. The same session cost hours to its own mistakes (M-19): it launched the generator without estimating its output size, the ~10 GB per cell of waveform evidence filled the disk and crashed Docker mid-run, and it ran 6 parallel ngspice jobs each defaulting to 8 OpenMP threads on an 8-core machine, which made every transient ~30x slower and timed out every characterization context with a message ("points failed settling/convergence") that reads like a circuit problem. Both fixes were measurements, not code: timing

one transient alone against the batch log, and `du` on the output tree. The lesson we kept: when an AI drives an unfamiliar tool, make it measure one unit (one simulation, one cell) before launching the batch.

8. Next time

Would hand to an AI: readers and writers of documented formats, test scaffolding, plots and tables from existing CSVs, glue scripts, first drafts of documentation, always with a numeric target from the handout to check against. **Would not:** choosing what a check compares against or its tolerance; deciding that a mismatch is acceptable; the estimate in the ranking (it passes every test while being wrong); anything in `tests/broken/`. And we would write the comparison with the handout's tables before asking for the code that produces them, because every serious mistake above was caught by such a comparison and missed by the code's own tests.